

Đề bài Tập lớn Môn Lập trình C: Xây dựng Hệ thống Quản lý Trung tâm Tin học

Mục tiêu: Vận dụng tổng hợp các kiến thức đã học (cấu trúc điều kiện, vòng lặp, hàm, mảng, con trỏ, struct, và cấp phát động) để xây dựng một ứng dụng console hoàn chỉnh.

Phần 1: Đặc tả Yêu cầu Chức năng Chi tiết

Chương trình phải phân quyền người dùng thành 3 vai trò: **Admin**, **Teacher** (Giáo viên), và **Student** (Học viên). Dữ liệu phải được quản lý bằng các mảng động (cấp phát bằng `malloc/realloc`) và tồn tại trong suốt thời gian chương trình chạy.

1.1. Chức năng Chung: Đăng nhập

- Đây là màn hình đầu tiên khi khởi động chương trình.
- Người dùng nhập tên đăng nhập và mật khẩu.
- **Yêu cầu kiểm tra (Validation):**
 - Kiểm tra Tồn tại: Tên đăng nhập có tồn tại trong danh sách tài khoản không?
 - Kiểm tra Mật khẩu: Nếu tên đăng nhập tồn tại, mật khẩu nhập vào có khớp không?
 - Thành công: Nếu cả hai đều đúng, chương trình phải đọc vai trò (Role) của tài khoản đó và chuyển hướng đến menu tương ứng (Admin, Teacher, hoặc Student).
- Chương trình phải tạo sẵn ít nhất 1 tài khoản Admin (ví dụ: `admin/admin`) để có thể đăng nhập lần đầu.

1.2. Menu Admin

Admin là người quản lý tài khoản và các khóa học của trung tâm.

1. **Danh sách tài khoản:**
 - Hiển thị danh sách TOÀN BỘ tài khoản (Admin, Teacher, Student).
 - Thông tin hiển thị: ID, tên đăng nhập, vai trò (Không hiển thị mật khẩu).
2. **Thêm tài khoản:**
 - Yêu cầu nhập tên đăng nhập, mật khẩu, vai trò (1=Admin, 2=Teacher, 3=Student).
 - **Kiểm tra:** Tên đăng nhập phải là duy nhất. Nếu trùng: "Lỗi: Tên đăng nhập này đã được sử dụng."
3. **Xóa tài khoản:**
 - Yêu cầu nhập tên đăng nhập của tài khoản cần xóa.

- **Kiểm tra:**

- Tài khoản admin gốc (ID = 1) không thể bị xóa.
- Admin không thể tự xóa chính tài khoản mình đang đăng nhập.

Hiển thị thông tin tài khoản bị xóa "ID , tên đăng nhập và vai trò"

- Hiển thị thông báo xác nhận (Y/N) trước khi xóa.
- *Nâng cao: Khi xóa Giáo viên, cần xử lý các Lớp học (ví dụ: gán ID giáo viên của lớp đó về 0 hoặc rỗng). Khi xóa Học viên, cần xóa các bản ghi đăng ký của họ.*

4. Quản lý Khóa học (CRUD):

- **Thêm Khóa học:** Nhập mã khóa học (duy nhất), tên khóa học, ID của giáo viên (phải tồn tại và có vai trò là Giáo viên), sĩ số tối đa. Sĩ số đã đăng ký sẽ tự động được gán là 0.
- **Sửa Khóa học:**

In ra menu mời bạn chọn thông tin muốn sửa

1. Tên khóa học
2. Id giáo viên (phải kiểm tra id tồn tại)
3. Sĩ số tối đa (không được nhỏ hơn sĩ số đã đăng ký hiện tại)

Sau đó cho phép nhập lại thông tin đã chọn

- **Xóa Khóa học:** Chỉ được xóa khi chưa có ai đăng ký (sĩ số đã đăng ký là 0).
- **Xem DS Khóa học:** Hiển thị mã khóa học, tên khóa học, ID giáo viên, sĩ số tối đa, sĩ số đã đăng ký. Sắp xếp giảm dần theo sĩ số tối đa

5. Đăng xuất: Quay lại màn hình Đăng nhập.

6. Thoát chương trình: Giải phóng toàn bộ bộ nhớ 3 mảng động.

1.3. Menu Teacher (Giáo viên)

Giáo viên quản lý các lớp học mình được phân công.

1. Xem lớp học của tôi:

- Lấy ID của giáo viên đang đăng nhập.
- Duyệt danh sách khóa học, in ra các khóa học có ID giáo viên khớp với ID của giáo viên này.

2. Xem danh sách học viên của lớp:

- Yêu cầu giáo viên nhập mã khóa học (phải là lớp của họ).
- Duyệt danh sách đăng ký, nếu mã khóa học khớp VÀ trạng thái là "Đang học", thì lấy ID học viên và tra cứu ngược để in ra Tên Học viên.

3. Cập nhật trạng thái hoàn thành:

- Yêu cầu nhập mã khóa học và ID học viên.
- Tìm bản ghi đăng ký có trạng thái "Đang học" khớp với cả hai.
- Nếu tìm thấy, đổi trạng thái thành "Đã hoàn thành".

- *Lưu ý: Thao tác này KHÔNG làm giảm số đã đăng ký của Lớp học.*
- 4. **Đăng xuất:** Quay lại màn hình Đăng nhập.
- 5. **Thoát chương trình:** Giải phóng bộ nhớ.

1.4. Menu Student (Học viên)

Học viên đăng ký và xem các khóa học.

1. **Xem danh sách tất cả Khóa học:**
 - Hiển thị mã khóa học, tên khóa học, số số tối đa, số số đã đăng ký.
 - Hiển thị thêm cột "Số chỗ còn lại" (tính bằng số số tối đa - số số đã đăng ký).
2. **Đăng ký Khóa học:**
 - Yêu cầu nhập mã khóa học muốn đăng ký.
 - **Yêu cầu kiểm tra:**
 1. Mã khóa học có tồn tại không?
 2. Học viên này đã đăng ký khóa này (và trạng thái là "Đang học") hay chưa?
 3. Lớp đã đầy chưa (số số đã đăng ký $\geq$ số số tối đa)?
 - Nếu tất cả kiểm tra qua:
 1. Tăng số số đã đăng ký của Khóa học đó lên 1.
 2. Thêm một bản ghi đăng ký mới (ghi lại ID học viên và mã khóa học, trạng thái là "Đang học").
3. **Xem các khóa học của tôi:**
 - Lấy ID của học viên đang đăng nhập.
 - Duyệt danh sách đăng ký.
 - Nếu ID học viên khớp, lấy mã khóa học và tra cứu ngược để in ra thông tin (Tên khóa học, trạng thái).
4. **Đăng xuất:** Quay lại màn hình Đăng nhập.
5. **Thoát chương trình:** Giải phóng bộ nhớ.

Phần 2: Gợi ý Tổ chức Code và Cấu trúc

Phần này đưa ra các gợi ý về cách tổ chức dữ liệu, quản lý trạng thái, và phân chia hàm để xây dựng các chức năng đã mô tả trong Phần 1. Bạn cần tự mình quyết định tên biến và cách triển khai chi tiết.

2.1. Cấu trúc dữ liệu (Structs) đề xuất

Bạn sẽ cần định nghĩa 3 cấu trúc (**struct**) chính để mô hình hóa 3 đối tượng của bài toán. Hãy suy nghĩ về các trường (fields) cần thiết cho mỗi cấu trúc.

1. **Cấu trúc Người dùng:**
 - Cần lưu trữ các thông tin để định danh duy nhất (ví dụ: ID), thông tin dùng để đăng nhập (tên đăng nhập, mật khẩu), và thông tin để phân quyền (vai trò).

2. Cấu trúc Khóa học:

- Cần lưu trữ mã định danh (ví dụ: mã khóa học), các thông tin mô tả (tên khóa học), thông tin liên kết (ID giáo viên nào phụ trách), và thông tin về sĩ số (tối đa, đã đăng ký).

3. Cấu trúc Đăng ký:

- Đây là cấu trúc liên kết, dùng để ghi lại lịch sử. Nó cần lưu trữ thông tin để biết *ai* (học viên nào) đã đăng ký *cái gì* (khóa học nào) và trạng thái của việc đăng ký đó (ví dụ: đang học, hay đã hoàn thành).

2.2. Các biến toàn cục (Quản lý trạng thái)

Để quản lý dữ liệu và trạng thái của chương trình, bạn sẽ cần các biến toàn cục.

• Quản lý dữ liệu:

- Bạn cần 3 con trỏ, mỗi con trỏ sẽ trỏ đến vùng nhớ của một mảng động (tương ứng với 3 `struct` ở trên).
- Bạn cần các biến đếm (kiểu số nguyên) để theo dõi số lượng phần tử *hiện tại* đang có trong mỗi mảng động.
- Bạn cần các biến (kiểu số nguyên) để tạo ID tự động tăng cho Người dùng và bản ghi Đăng ký, để đảm bảo ID mới luôn là duy nhất.

• Quản lý trạng thái:

- Bạn cần một biến (có thể là một `struct` hoặc một `int`) để lưu trữ thông tin của người dùng *sau khi* họ đăng nhập thành công.
- Bạn cần một biến cờ (flag) để chương trình biết đang ở trạng thái "đã đăng nhập" hay "chưa đăng nhập".

2.3. Cấu trúc Hàm (Module hóa)

Bạn nên chia chương trình thành nhiều hàm, mỗi hàm thực hiện một nhiệm vụ cụ thể để code dễ đọc và dễ bảo trì.

• Nhóm hàm quản lý luồng chính:

- Cần một hàm để xử lý logic đăng nhập.
- Cần các hàm riêng biệt để hiển thị và xử lý cho từng menu (Admin, Teacher, Student).
- Cần một hàm để khởi tạo dữ liệu ban đầu (ví dụ: tạo tài khoản 'admin' gốc).
- Cần một hàm để giải phóng toàn bộ bộ nhớ đã cấp phát trước khi thoát chương trình.

• Nhóm hàm nghiệp vụ:

- Mỗi chức năng chi tiết trong Phần 1 nên được viết thành một hàm riêng. Ví dụ: một hàm cho "Thêm tài khoản", một hàm cho "Xóa tài khoản", một hàm cho "Đăng ký khóa học", v.v.

• Nhóm hàm tiện ích (Hàm hỗ trợ):

- Đây là các hàm dùng chung, được gọi bởi nhiều hàm nghiệp vụ khác nhau để tránh lặp code.
- Ví dụ:
 - Một hàm tìm kiếm người dùng dựa trên tên đăng nhập.
 - Một hàm tìm kiếm người dùng dựa trên ID.
 - Một hàm tìm kiếm khóa học dựa trên mã khóa học.
 - Một hàm tìm bản ghi đăng ký đang học của một sinh viên cho một khóa học cụ thể.
 - Một hàm tiện ích để xử lý việc nhập liệu (như xóa bộ đệm `stdin`).
 - Một hàm tiện ích để hỏi người dùng xác nhận "Bạn có chắc không? (Y/N)".

Phần 3: Hướng dẫn Kiến thức C bổ sung

(Phần này giống hệt Assignment 1, vì các kỹ thuật này là chung cho mọi dự án C console)

3.1. Kỹ thuật Nhập/Xuất Nâng cao

a. Nhập chuỗi có dấu cách (ví dụ: Tên Khóa học)

- **Vấn đề:** `scanf("%s", ...)` sẽ dừng khi gặp dấu cách.
- **Giải pháp:** Dùng `fgets` để đọc cả dòng.

Cách xử lý `\n`:

```
#include <string.h>
char tenKhoaHoc[100];
printf("Nhap ten khoa hoc: ");
fgets(tenKhoaHoc, 100, stdin);
tenKhoaHoc[strcspn(tenKhoaHoc, "\n")] = '\0';
```

•

b. Vấn đề "Trôi lệnh" (Lỗi `scanf` và `fgets`)

- **Vấn đề:** `scanf("%d", ...)` để lại `\n` trong bộ đệm. `fgets` tiếp theo sẽ đọc `\n` đó.

Giải pháp: Viết hàm `xoaBoDem()` và gọi nó sau mỗi `scanf`.

```
void xoaBoDem() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}
// ...
scanf("%d", &luaChon);
xoaBoDem(); // Gọi ngay sau scanf
```

```
fgets(tenKhoaHoc, 100, stdin); // OK
```

•

c. In theo format (In dạng bảng)

Dùng `%-10s` (căn trái) và `%10s` (căn phải) để căn chỉnh các cột.

```
printf("%-10s | %-30s | %-10s | %-10s\n", "Ma KH", "Ten Khoa Hoc", "Si So", "Con Lai");
printf("-----\n");
printf("%-10s | %-30s | %-10d | %-10d\n",
       kh.maKhoaHoc, kh.tenKhoaHoc,
       kh.siSoToiDa, kh.siSoToiDa - kh.siSoDaDangKy);
```

•

3.2. Xử lý chuỗi (Thư viện `string.h` và `ctype.h`)

- **So sánh chuỗi:** Dùng `strcmp()` (trả về 0 nếu giống hệt). `if (strcmp(matKhanhNhap, nguoiDung.matKhanh) == 0) { ... }`
- **Tìm chuỗi con:** Dùng `strstr()` (trả về NULL nếu không tìm thấy). `if (strstr(khoaHoc.tenKhoaHoc, keyword) != NULL) { ... }`

3.3. Kỹ thuật Sắp xếp mảng Struct

- Dùng thuật toán (ví dụ: Bubble Sort) để sắp xếp danh sách khóa học dựa trên một trường (ví dụ: sĩ số đã đăng ký).

Khi hoán vị, bạn phải hoán vị toàn bộ `struct`, không phải chỉ hoán vị con trỏ.

// Cấu trúc hoán vị

```
struct KhoaHoc temp = dsKhoaHoc[j];
dsKhoaHoc[j] = dsKhoaHoc[j+1];
dsKhoaHoc[j+1] = temp;
```

•

3.4. Xóa phần tử khỏi mảng động

- Áp dụng khi "Xóa tài khoản" hoặc "Xóa Khóa học".
- **Giải pháp:**
 1. Tìm vị trí (index) của phần tử cần xóa.
 2. Dùng vòng lặp `for` để dời tất cả các phần tử phía sau lên 1 vị trí (ghi đè lên phần tử cần xóa).
 3. Giảm biến đếm (ví dụ: số lượng người dùng).
 4. Gọi `realloc` để thu nhỏ mảng.

```
// Giả sử xóa tại 'indexCanXoa'
for (int i = indexCanXoa; i < soLuongNguoiDung - 1; i++) {
    dsNguoiDung[i] = dsNguoiDung[i + 1];
}
soLuongNguoiDung--;
// Cấp phát lại
struct NguoiDung* tempPtr = realloc(dsNguoiDung, soLuongNguoiDung * sizeof(struct
NguoiDung));
// ... kiểm tra tempPtr ...
```